

Class starts in 5 minutes

Agenda

↳ Semaphores

↳ Atomic Integers -

↳ Concurrent Hash Maps }

↳ Deadlocks. =

Showcase

remove() →



of consumers allowed = no. of filled slots

of producers allowed = no. of empty slots

mutex (lock) ⇒ slow

Semaphores

↳ Consider it as mutex with an upper bound ≥ 1

→ Lock lock = new —

Semaphore s = new Semaphore(2)

↳ Means two threads can take the lock at the same time.

new Semaphore(x)

↳ x threads can take a lock at the same time

2 imp semaphore: ←

① semaphore.acquire() ←
 semaphore.release() ←

lock.unlock() ←

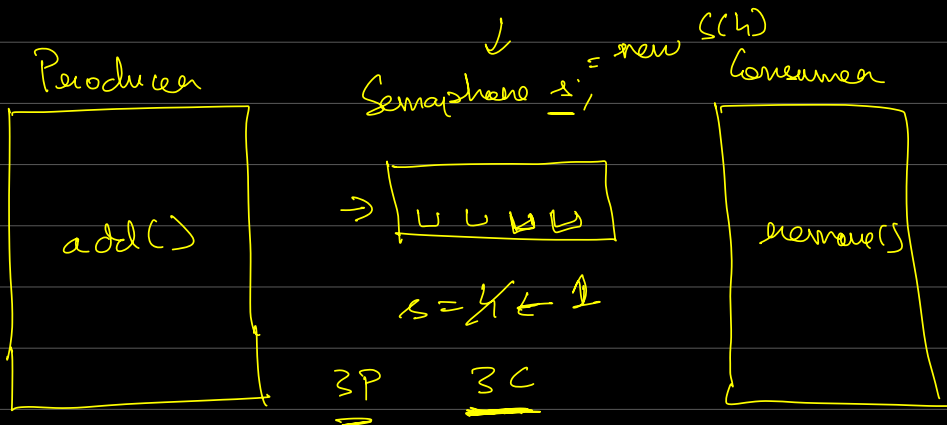
② semaphore.release() ←

Semaphore s = new Semaphore(1);
 s.release() → +1 -
 s.acquire() → -1

Semaphores → integer + mutex

Semaph s = new(1) = mutex

Sol 1)

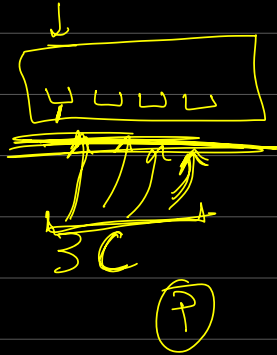


[Problem is caused because we are using only one semaphore object for both producer and consumer]

Sol 2)

① Semaphore P = new S(4) 3 ←
 Producer ② Semaphore C = new S(0) Consumer ←

→ P.acquire()
 → add() ←
 C.release() -

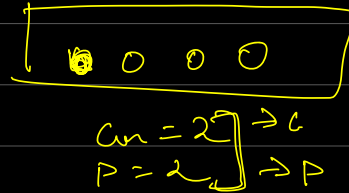


C.acquire() ←
 remove() ←
 P.release() =

maxSize = 4

→ Consumer = 0 1

producer = 4 3 2 1 2



<u>P1</u>	<u>P2</u>	<u>P3</u>	<u>C1</u> ←
① P.acquire()	② P.acquire()	③ P.acquire()	④ C.acquire() ←
④ add()	⑤ add() ←		wait x
	⑦ C.release()		⑤ remove()
			⑥ P.release()

Producer is notifying consumer that it has done its job, now you can do yours & vice versa.

$\text{maxSize} = 4$
 $\rightarrow \text{consumer} = \emptyset$
 $\text{producer} = K \quad \cancel{2} \quad \cancel{1} \quad \cancel{0} \quad \boxed{0 \quad 0 \quad 0}$

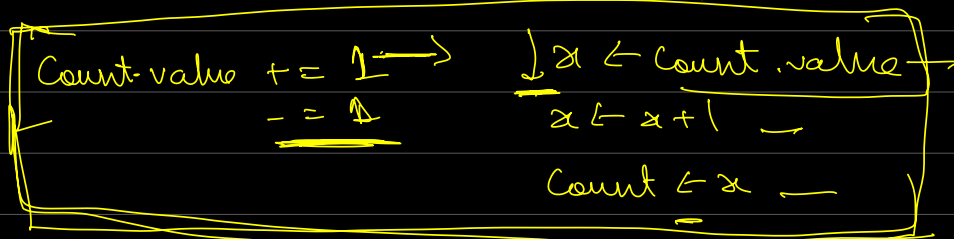
<u>c1</u>	<u>c2</u>	<u>p1</u>	<u>p2</u>	<u>p3</u>	<u>c3</u>	<u>p4</u>
c.acquire()	c.acquire()	p.acquire()	p.acquire()	p.acquire()	c.acquire()	c.del()
<u>wait()</u>	<u>wait()</u>	<u>add()</u>	<u>add()</u>	<u>add()</u>	<u>wait()</u>	<u>c.release()</u>
<u>remove()</u>	<u>remove()</u>	<u>c.release()</u>	<u>c.release()</u>		<u>remove()</u>	
<u>producer()</u>	<u>producer()</u>					

10:20 → 10:26 (6 minutes)

Concurrent Data Types

Adder Subtractor

Atomic Integer
Done in one go.



Atomic Operations ⇒ that are executed in one go.

Integer ⇒ Atomic Integer ←
Boolean ⇒ " Boolean

Integer
x + 1

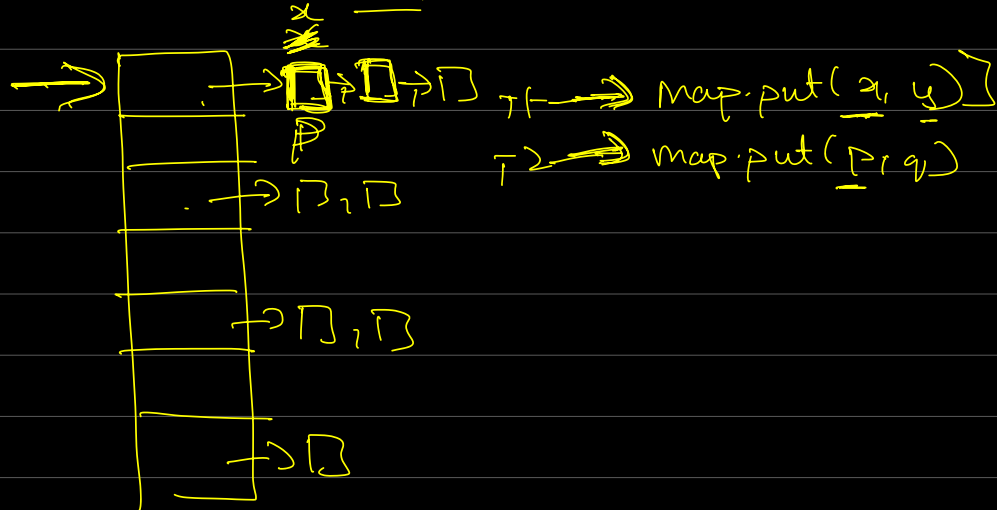
~~A.P.~~

Atomic Integer ⇒ Provide atomic methods to
perform common ops on integer

→ Atomic Integer ⇒ This comes with performance overhead.
→ CAS ⇒ Compare and Swap

Concurrent Hash Maps

Map < String, String > map = new HashMap();

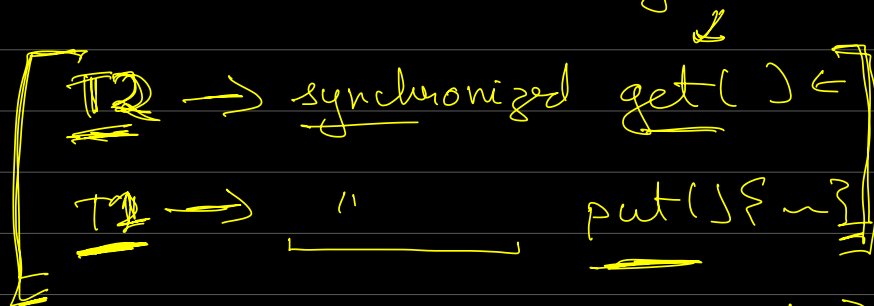


HashMaps are not thread safe

→ are like HashMaps but all methods are synchronized

HashTables are thread safe.

↳ All the methods are synchronized.



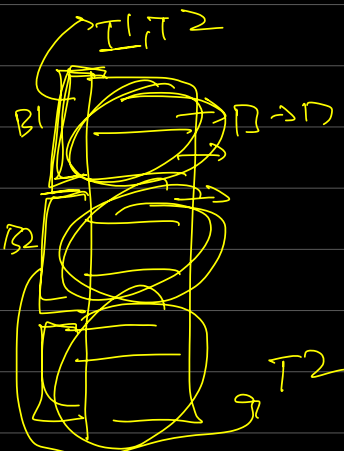
Map m = new ConcurrentHashMap < > ();

map.put(-, -) ← T1

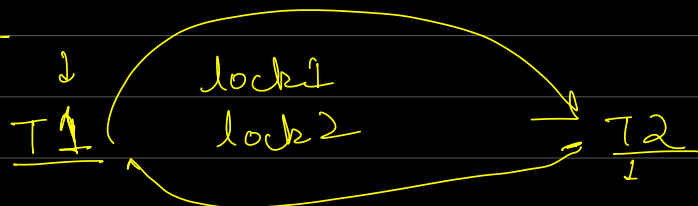
map.put(-, -) ← T2

get(-, -) ← T3

get(-, -) ← T4



Deadlock



- ① lock1.lock()
- ④ lock2.lock() -
 ↳ wait()
- ② lock2.lock()
- ③ lock1.lock() ←
 ↳ wait()

Deadlock ⇒ A situation when the complete system comes to a halt because threads are waiting on locks that they will never be able to get.

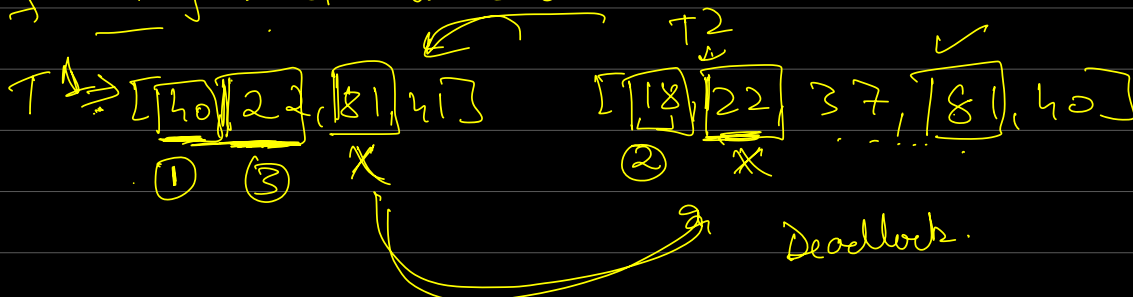
[Prevention is better than cure]

Solutions :

- ① Deadlock Prevention ← → Clean env.
- ② Deadlock recovery ← → Take medication
- ③ Deadlock ignore ← → Ignore the disease.

Deadlock avoidance

↳ Taking locks in order →



T1
 $\{22, 40, 41, 81\}$
✓ ✓ ✓ ✓

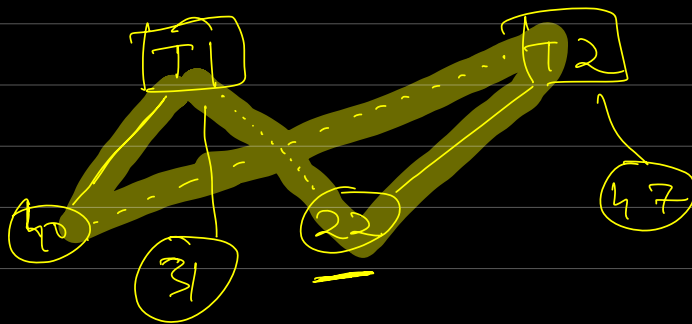
T2
 $[18, 22, 37, 40, 81]$
~~✓~~ ~~✓~~ ✓ ✓

Deadlock Identification

① Wait for timeout

↳ If your thread is waiting for 'x' seconds, kill the thread.

② detecting a cycle in the graph



As soon as you detect a cycle
 $\Rightarrow$ deadlock:
 $\Rightarrow$ kill T2 & instant +2.

After Identification

① Recovery

↳ Kill the thread.

② Ignoreance $\Rightarrow$ Very common in O.S.

Production systems

(1) Identify deadlocks using timeouts $\leftarrow$

(2) $\text{tryLock()} \longleftrightarrow$ If I don't get a lock, I will
lock Kill the thread & exit.

(3) If I don't get a lock in 'x' seconds, return
on try again.